

AI-Powered SRE Agents for Azure Container Apps (2025–Present)

Problem: Azure Container Apps incidents required manual root cause analysis, leading to long mitigation and resolution times for customer-facing issues.

What I Did:

- Designed and deployed AI-powered diagnostic agents for ACA incident resolution — ICM Summarization Agent, CoreDNS Agent, Sessions Agent, and Billing Subagent.
- Shifted architecture from workflow-based to investigative prompts, dramatically improving AI-aligned RCA accuracy; closed 20+ agent backlog items through targeted ICM analysis.
- Evolving agent toward autonomous auto-repair — automatically comparing agent-generated RCA vs actual RCA, creating repair items, and auto-resolving qualifying incidents.
- Enabled Foundry Control Plane (FCP) support so all agents created on ACA are visible on the Foundry Control Plane.
- Presented the RCA Agent at the IDC Pinnacle event, showcasing its impact on incident diagnostics and reliability.

Impact: Reduced ACA incident P50 time-to-mitigate by 97% (185 → 6 min) and P50 time-to-resolve by 64% (269 → 98 min). Targets reduction of ~99 monthly CRIs through accelerated root cause analysis.

Azure Container Apps Sessions (2024–2025)

Problem: ACA Sessions lacked observability, had critical async execution bugs, and needed data-plane APIs for internal and external customers.

What I Did:

- Built Session Management APIs (Delete, List, Get) to improve session observability and control.
- Fixed critical async execution bugs in code interpreter sessions and Node.js session-specific issues.
- Enabled early session deletion — a key requirement from the Fabric Team — reducing billing and costs.
- Added performance tests for ACA Sessions, identifying code execution regressions.
- Enabled .NET code interpreter support for the Security Copilot Team.
- Implemented Geneva actions to manage upgrades for Node.js sessions.

Impact: Improved session stability and experience for ~300 external customers actively using ACA Sessions. Enabled cost optimization for ACA session customers through early deletion support.

Azure Functions on Azure Container Apps (2025–Present)

Problem: Azure Functions on ACA lacked feature parity with native Azure Functions, missing key management, diagnostics, and scaling capabilities.

What I Did:

- Implemented function keys management commands for ACA Functions.
- Added invocation tracing/summary and durable-events logging for diagnostics.
- Authored API spec for Function ARM APIs.
- Added support for editing KEDA scaling rules in Azure Functions on ACA.
- Implemented drain functionality for Azure Functions on ACA.
- Working on closing feature parity gaps between Azure Functions on ACA and native Azure Functions.

Impact: Enhancements benefit ~1,322 external ACA Functions apps, improving security, observability, and operational efficiency.

Open-Source Contributions to Dapr — CNCF Project (2021–2024)

Problem: Dapr, a CNCF open-source project for distributed applications, needed components upgraded to stable, new features, improved developer experience, and active community support.

What I Did:

- Upgraded MQTT PubSub component from alpha to stable; contributed fix to upstream library; added certification tests.
- Marked Azure CosmosDB state store, Configuration API building block, and Redis/PostgreSQL configuration stores to stable.
- Implemented cascade terminate and purge for Dapr Workflows — changes spanning runtime and multiple SDKs.
- Added bulk subscribe support in Azure Event Hubs and Dapr Workflow performance tests; identified several P0 bugs.
- Enabled airgap installation support; built MSI installer; made Dapr available in winget with automated package updates.
- Published combined coverage reports for conformance and certification tests.
- Resolved customer issues across GitHub (Runtime, Components-Contrib, SDKs) and Discord.

Impact: Accelerated Dapr component maturity and improved developer experience, driving broader adoption of Dapr across Azure and the open-source community.

Dapr Internal Fork & CI/CD Infrastructure (2021–2024)

Problem: Microsoft's internal services (AKS, Container Apps) needed a reliable internal fork of Dapr with fast release turnaround, compliance, and multi-architecture support.

What I Did:

- Architected and maintained Dapr internal fork across 5 repositories (dapr, cli, dashboard, components-contrib, kit) with build, code-signing, and release pipelines.
- Automated OSS-to-internal merge pipeline, achieving internal releases within <1 day SLA of OSS release.
- Re-designed internal repository to a patch-based model with separate per-service images for faster, targeted releases.
- Migrated CI/CD pipelines to 1ES (OneBranch) for compliance; integrated Ev2 deployment for Standard Deployment Practices.
- Added multi-arch support (ARM64, ARM, AMD64), Mariner and distroless images; handled 1P Container Registry migration.
- Delivered nightly builds for Azure Container Apps.

Impact: Enabled AKS and Container Apps to reliably consume internal Dapr images with rapid turnaround, supporting seamless adoption of Dapr across Azure distributed services.

Dapr AKS Extension — Security & Multi-Region Rollouts (2022–Present)

Problem: Dapr AKS Extension customers needed secure, up-to-date versions across all Azure regions with fast CVE remediation and reliable rollouts.

What I Did:

- Led Dapr 1.15 and 1.16 rollouts across all Azure regions.
- Fixed multiple CVE/security vulnerabilities; remediated issues across ~400 AKS clusters.
- Expanded AKS extension availability to new regions based on customer demand.
- Integrated CodeQL into AKS Extension pipeline for security and code quality.
- Consolidated multiple pipelines into a single OneBranch-compliant pipeline to expedite releases.
- Resolved customer-reported AKS extension ICMs promptly.

Impact: Upgraded ~260 subscriptions and ~400 AKS clusters to secure Dapr versions, strengthening security posture and ensuring reliable Dapr experiences for enterprise customers.

On-Call Reliability & DRI Operations (2024–Present)

Problem: Azure Container Apps required consistent, high-quality on-call coverage to maintain service uptime and resolve customer-impacting incidents rapidly.

What I Did:

- Served as on-call DRI, resolving 200+ incidents including 27+ CRIs, outperforming team baselines.
- Handled 36 Sev2 LSIs (P50 TTM 1h, P50 TTR 25h vs ACA baseline 3.9h/60h) and 6 Sev3 CRIs.
- Delivered repair items to address root causes, reducing incident recurrence and improving system resilience.
- Improved alert fidelity by updating 5 LSI alerts based on investigation findings.

Impact: Ensured high service uptime and reduced operational noise through proactive issue resolution, consistently exceeding ACA on-call baselines.

Azure Key Vault Integration & Security Posture (2025)

Problem: ACA Managed Environment Storage API was handling raw connection strings in ~27K daily API calls, creating a security risk.

What I Did:

- Integrated Azure Key Vault with Managed Environment Storage API, enabling secure secret handling.
- Eliminated raw connection strings in storage create API calls.
- Resolved CodeQL violations across container apps, improving code security and compliance.

Impact: Provided a secure alternative to raw account keys for ~27K daily storage API calls, strengthening security posture and aligning with enterprise-grade standards.